CC232 - Algoritmos y Estructuras de Datos

Práctica Calificada 3 — Semanas 5 a 7

Universidad Nacional de Ingeniería — Ciclo 2026-2

Duración	150 minutos	**Puntaje**	20 puntos
Modalidad	individual, presencial, lápiz y papel	**Alcance**	Semanas 5 a 7

Nombre: _____ **Código:** _____

Indicaciones:

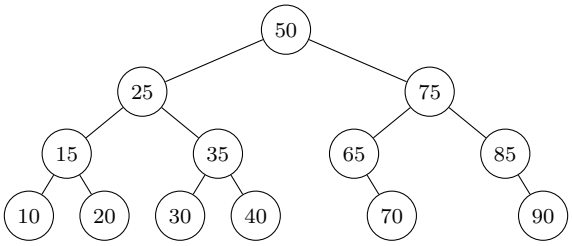
- **Regla general de calificación:** esta regla se aplica a todos los problemas e incisos y no se repetirá en cada uno. Si la respuesta omite la evidencia solicitada, el inciso obtiene 0. Esto incluye, según corresponda, resultados o complejidades sin justificación, gráficos sin explicar qué representan y por qué son válidos, pseudocódigo o Java simplificado sin explicación de los pasos, explicación algorítmica sin el pseudocódigo solicitado, y conceptos técnicos mencionados sin relacionarlos con el estado y las operaciones concretas del problema.
- Cuando indiques una complejidad, explica qué trabajo depende de n o de h y **separa localización de modificación** cuando corresponda.
- Antes de modificar una estructura, conserva las referencias necesarias y verifica los invariantes al terminar. No uses Java Collections.

Problema 1. BST: invariante de parent, eliminación con `splice` y sucesor — 4 puntos

Semana 5. Un índice en memoria mantiene claves en un BST sin balanceo automático. Cada nodo posee referencias `left`, `right` y `parent`. La eliminación se realiza mediante la operación auxiliar `splice(Node u)` y el sucesor inorder:

```
void splice(Node u) { // precondicion: u.left == null || u.right == null
    Node s = (u.left != null) ? u.left : u.right;
    Node p = u.parent;
    if (u == root) root = s;
    else if (p.left == u) p.left = s;
    else p.right = s;
    if (s != null) s.parent = p;
}
```

El árbol contiene inicialmente el siguiente conjunto de claves ($n = 12$):



Se ejecutan, en este orden: `remove(10)`, `remove(65)`, `remove(25)` y `remove(50)`.

a) Realiza la traza gráfica de las cuatro operaciones. Para cada una indica: el caso de eliminación (hoja, un hijo o dos hijos), el nodo que se desvincula físicamente mediante `splice`, la sustitución de clave lógica (si aplica) y dibuja el árbol resultante tras cada llamada. **(1,5 puntos)**

b) Para `remove(25)`, detalla la secuencia exacta de modificaciones de punteros en memoria (`parent`, `left`, `right`) y el valor copiado. Explica formalmente por qué el sucesor inorder w de cualquier nodo con dos hijos nunca puede tener hijo izquierdo ($w.left == null$), pero sí hijo derecho, y cómo responde `splice(w)`. **(0,75 puntos)**

c) Un estudiante omite la última línea de `splice` (`if (s != null) s.parent = p;`). Considera que se ejecuta esa versión defectuosa en `remove(65)`. ¿El recorrido inorder posterior sigue siendo correcto? Identifica qué referencia queda rota y describe exactamente qué ocurrirá si después se invoca `remove(70)`. **(0,75 puntos)**

d) Separa de forma explícita el costo de localización (`findLast`) del costo de modificación (`splice` y reasignación) en función de h. Explica por qué es un error grave escribir automáticamente $O(\log n)$ y dibuja un árbol degenerado de n nodos donde `remove` cueste $\Theta(n)$. **(1,0 punto)**

Problema 2. AVL: los cuatro casos de rotación y orden de actualización — 4 puntos

Semana 6. Un conmutador de paquetes mantiene rutas en un árbol AVL donde cada nodo tiene campos `x`, `height` (h(hoja) $= 1, h$(null) $= 0$), `left` y `right` (sin puntero parent). Se define `balanceFactor(u)` = `height(u.left)` − `height(u.right)`. Invariante AVL: $|$`balanceFactor(u)`$| \leq 1$ para todo nodo u. Las rotaciones se implementan como:

```
Node rotateRight(Node y) {
    Node x = y.left; Node middle = x.right;
    x.right = y; y.left = middle;
    updateHeight(y); // Paso 1
    updateHeight(x); // Paso 2
    return x;
}
Node rotateLeft(Node x) {
    Node y = x.right; Node middle = y.left;
    y.left = x; x.right = middle;
    updateHeight(x); updateHeight(y);
    return y;
}
```

A partir de un árbol AVL inicialmente vacío, se insertan en este orden las siguientes claves:

`45, 20, 10, 30, 25, 60, 80, 70`

a) Realiza la traza paso a paso de cada inserción. Para cada inserción que provoque una violación del invariante AVL:

I. Identifica el nodo más profundo (el primero detectado en el retorno recursivo) donde $|FE| = 2$.

II. Clasifica el caso (LL, RR, LR o RL) justificando con el factor de balance del nodo y de su hijo.

III. Dibuja el subárbol afectado antes y después de aplicar las rotaciones simples o dobles.

IV. Dibuja el árbol completo final tras insertar las 8 claves. **(1,5 puntos)**

b) Analiza críticamente `rotateRight(Node y)`. Explica con rigor por qué es imprescindible ejecutar `updateHeight(y)` **antes** que `updateHeight(x)`. ¿Qué valor numérico erróneo de altura adoptaría x si un programador invirtiese el orden de esas dos líneas? Demuéstralo trazando un subárbol concreto. **(0,75 puntos)**

c) Un estudiante afirma: *"Si un nodo tiene $FE = +2$, la anomalía siempre está hacia la izquierda, por lo que basta ejecutar una única `rotateRight(u)` directa sin consultar el signo de su hijo izquierdo"*. Aplica su algoritmo erróneo a la inserción de la clave `25` (caso LR): dibuja el árbol resultante, calcula sus factores de balance y demuestra que el resultado no es un AVL válido. **(0,75 puntos)**

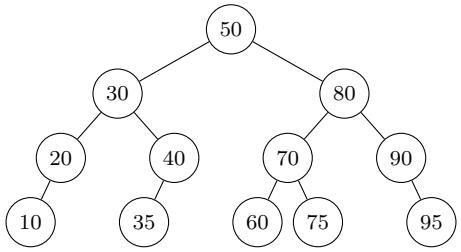
d) Construye una tabla con los campos `x`, `height` y `balanceFactor` para cada uno de los 8 nodos del árbol AVL final obtenido en el inciso a). **(0,5 puntos)**

e) Sea $N(h)$ la mínima cantidad de nodos que puede contener un AVL válido de altura h, con $N(1) = 1, N(2) = 2$ y $N(h) = 1 + N(h-1) + N(h-2)$ para $h \geq 3$. Calcula $N(h)$ hasta $h = 6$. A partir de estos valores, deduce la altura máxima posible de un AVL con $n = 8$ nodos y compárala con la altura máxima de un BST sin balanceo con 8 nodos. **(0,5 puntos)**

Problema 3. AVL: eliminación con rebalanceo en cascada y poda en rango — 4 puntos

Semanas 5 y 6. A diferencia de la inserción en un AVL (donde a lo sumo una rotación simple o doble rebalancea todo el árbol), la eliminación de un nodo puede reducir la altura de un subárbol y desencadenar desbalances sucesivos en múltiples niveles ascendentes hacia la raíz ($O(\log n)$ rotaciones).

Considere el siguiente árbol AVL balanceado ($n = 11$, con $h = 4$):



a) Sobre el árbol dado, se ejecuta la eliminación de la clave `10` (`remove(10)`):

I. Realiza la traza del retroceso recursivo actualizando alturas y factores de balance desde el punto de desvinculación.

II. Identifica el primer nodo donde se produce desbalance, clasifica el caso (LL, RR, LR o RL) y describe la rotación ejecutada.

III. Muestra si dicho rebalanceo local propaga o no un desbalance a sus ancestros superiores (incluyendo la raíz).

IV. Dibuja el árbol balanceado final. Explica por qué, en el peor caso, la eliminación en AVL es $O(\log n)$ en cantidad de rotaciones mientras que la inserción es $O(1)$. **(1,25 puntos)**

b) Diseña e implementa en Java simplificado o pseudocódigo riguroso el método recursivo:

```
public Node deleteMax(Node u)
```

El método debe localizar y desvincular el elemento con el valor máximo del subárbol enraizado en u, actualizar correctamente las alturas de los ancestros en el retorno recursivo, aplicar `rebalance(u)` en cada nivel y retornar la nueva raíz local del subárbol. Especifica de forma explícita precondición y postcondición. **(1,25 puntos)**

c) Demuestra formalmente el costo temporal en el peor caso de `deleteMax(Node u)` en función de n. Explica por qué el diseño recursivo no requiere punteros al padre (`parent`) para restaurar el balance. **(0,75 puntos)**

d) Se requiere una operación `int countInRange(Node u, int low, int high)` que retorne la cantidad de claves del AVL dentro del intervalo cerrado $[low, high]$. Diseña el algoritmo con poda inteligente: no desciendas a subárboles que no puedan contener elementos dentro del rango. Explica qué poda realizas y por qué su costo en el mejor caso es $O(\log n)$ a diferencia del $O(n)$ de un recorrido inorder ciego. **(0,75 puntos)**

Problema 4. BinaryHeap: `heapify` $O(n)$, operaciones y forma — 4 puntos

Semana 7. Un sistema de triaje hospitalario gestiona pacientes con un `BinaryHeap` (Min-Heap: menor valor numérico representa mayor gravedad). Se implementa sobre un arreglo secuencial `Integer[] a` y un contador `n`. Relaciones sobre índices base 0: `left(i)` $= 2i+1$, `right(i)` $= 2i+2$, `parent(i)` $= \lfloor (i-1)/2 \rfloor$. Invariantes: (1) **Orden:** `a[parent(i)]` $\leq$ `a[i]` para todo $0 < i < n$; (2) **Forma:** los elementos ocupan consecutivamente `a[0..n-1]`.

Se recibe el siguiente arreglo desordenado de $n = 10$ prioridades:

`a =`

0	1	2	3	4	5	6	7	8	9
22	14	35	10	8	28	40	5	12	6

`n = 10, a.length = 16`

a) Ejecuta la construcción del heap mediante el algoritmo de Floyd: `heapify()`.

I. Indica el índice del primer nodo interno evaluado ($i = \lfloor n/2 \rfloor - 1$) y explica por qué se omiten los nodos con índices mayores.

II. Detalla para cada valor de i (desde $\lfloor n/2 \rfloor - 1$ hasta 0): los hijos comparados, los intercambios (`swap`) efectuados por `trickleDown(i)` y el contenido del arreglo tras finalizar cada paso.

III. Dibuja el árbol binario casi completo final y cuenta la cantidad exacta de intercambios ejecutados. **(1,5 puntos)**

b) Sobre el Min-Heap obtenido en el inciso a), ejecuta en este orden:

I. `remove()`: extracción del paciente más prioritario.

II. `remove()`: extracción del segundo paciente.

III. `add(4)`: ingreso de una nueva emergencia con prioridad 4.

Para cada operación, indica los elementos comparados, la trayectoria de desplazamiento (`trickleDown` o `bubbleUp`), el valor retornado (en `remove`) y el estado físico final del arreglo con su nuevo valor de `n`. **(1,0 punto)**

c) Demuestra analíticamente por qué `heapify()` construye el montículo en tiempo $O(n)$, mientras que insertar los n elementos uno tras otro con llamadas a `add(x)` requiere $O(n \log n)$ en el peor caso. Formula la sumatoria por niveles y demuestra su convergencia asintótica:

$$T(n) = \sum_{k=0}^{\lfloor \log_2 n \rfloor} \left\lceil \frac{n}{2^{k+1}} \right\rceil O(k) \leq O(n)$$

(0,75 puntos)

d) Un practicante propone optimizar `remove()` ahorrándose subir el último elemento a la raíz: sugiere reemplazar la raíz vacía por el menor de sus hijos (`a[0]` $=$ mín(`a[1]`, `a[2]`)) y repetir ese corrimiento hacia abajo hasta dejar un hueco en una hoja. Explica detalladamente qué invariante de estructura se destruye con esta propuesta y demuestra por qué las fórmulas $2i+1$ y $2i+2$ dejarían de mapear los nodos correctamente en operaciones futuras. **(0,75 puntos)**

Problema 5. Cuando un flujo ordenado de Heap degrada un BST y por qué AVL lo salva — 4 puntos

Semanas 5, 6 y 7. En un sistema de analítica de eventos, se procesan datos dinámicos comparando el comportamiento de un **BST sin balanceo**, un **Árbol AVL** y un **BinaryHeap (Min-Heap)**.

a) Degradación topológica en BST: Supón que se tienen n elementos en un Min-Heap válido. Se extraen los n elementos mediante n invocaciones sucesivas a `remove()`, y a medida que salen, cada elemento se inserta de inmediato con `add(x)` en un árbol BST (inicialmente vacío) sin balanceo automático.

I. ¿En qué orden numérico relativo emergen los elementos desde el Min-Heap?

II. Describe con precisión la topología del árbol BST resultante (forma geométrica y ramificación).

III. Indica la altura exacta $h(n)$ que alcanzará dicho BST en términos de n.

IV. ¿A qué complejidad asintótica degeneran `contains(x)` y `remove(x)` sobre este BST? Explica por qué este escenario representa el peor caso absoluto para un BST. **(1,25 puntos)**

b) Inmunidad y rebalanceo sistemático en AVL: Si la misma secuencia de elementos extraídos del Min-Heap se inserta consecutivamente en un árbol AVL inicialmente vacío:

I. ¿Qué tipo específico de desbalance (LL, RR, LR o RL) se detectará repetitivamente durante la inserción?

II. ¿Qué rotación elemental resuelve cada uno de estos desbalances?

III. Demuestra la cota superior estricta de la altura del AVL resultante y explica cómo el rebalanceo neutraliza la degradación observada en el BST. **(1,0 punto)**

c) La falacia de la búsqueda en Heap vs AVL: Un ingeniero afirma: *"Dado que tanto el BinaryHeap como el árbol AVL son árboles con altura $O(\log n)$, la operación `contains(x)` para determinar si una clave arbitraria x existe toma $O(\log n)$ en ambos casos"*. Refuta formalmente la afirmación. Explica la diferencia entre el orden parcial vertical del heap y el orden total horizontal del árbol de búsqueda. Demuestra cuál es el costo real de `contains(x)` en un BinaryHeap de n elementos en el peor caso. **(0,75 puntos)**

d) Matriz de decisión de ingeniería: Completa la tabla indicando la complejidad en el peor caso ($O(1)$, $O(\log n)$, $O(n)$ u $O(n \log n)$):

Operación de ingeniería	**BST sin balanceo**	**Árbol AVL**	**BinaryHeap (Min)**
Construcción a partir de n elementos desordenados			
Extracción del elemento mínimo (`removeMin`)			
Búsqueda de una clave arbitraria (`contains(x)`)			
Emisión de todas las claves ordenadas de menor a mayor			

Fundamenta qué estructura elegirías en cada escenario:

- **Escenario Alpha:** Carga de 95 % inserciones y extracciones del mínimo, y 5 % reportes batch.
- **Escenario Beta:** Carga de 70 % búsquedas puntuales (`contains`) y 30 % consultas de rango (`countInRange`). **(1,0 punto)**

Distribución sugerida del tiempo:

Problema	**Tema y estructura de datos**	**Puntaje**	**Tiempo sugerido**
Problema 1	BST: Eliminación con `splice`, sucesor y punteros `parent` (Semana 5)	4,0 pts	30 min
Problema 2	AVL: Inserción, rotaciones múltiples y orden de actualización de alturas (Semana 6)	4,0 pts	30 min
Problema 3	AVL: Eliminación en cascada, método recursivo `deleteMax` y poda (S6, S5)	4,0 pts	30 min
Problema 4	BinaryHeap: `heapify` $O(n)$, colas de prioridad e invariante de forma (Semana 7)	4,0 pts	30 min
Problema 5	Integrador: Coexistencia, flujos ordenados y matriz de decisión (S5, S6, S7)	4,0 pts	25 min
Revisión	Comprobación de invariantes de enlaces y casos frontera	—	5 min
	Total de la evaluación	**20,0 pts**	**150 min**

Criterio general: Se evalúa la coherencia entre ADT, representación, algoritmo, invariantes y complejidad, de acuerdo con la regla general de calificación indicada al inicio.